SIMATS ENGINEERING

Department of _____

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Particulars	Details
Student Name	__Pavithra Sri S___
Register Number	___192572079_____
Department / Section	___CSE with AI_____
Academic Year / Semester	_____2026_____
Faculty Name	___Amirtha_____
Nature of Assignment	Individual
Duration	1 Week
Total Marks	50
Date of Submission	_____
GitHub Repository	DSA01-<RegisterNumber>-Assignment1
Compiler	g++ (C++17)

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Impact Analysis	2		
12B	Evaluation of Original Design	2		
12C	Adaptation or Redesign	2		
12D	Evidence-Based Defence	2		
	TOTAL	50		

OVERALL REMARKS

Student Signature: ___Pavithra Sri S___ Faculty Signature: _____

Program Director: _____

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – CO1

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Nature: Individual Duration: 1 Week Total Marks: 50 Bloom's Level: K5 – Evaluate

Scenario and Working Assumptions

The metro has Inner, Middle and Outer zones with base fares of Rs 10, Rs 20 and Rs 30. A 20% surcharge applies during 08:00–09:59 and 17:00–19:59. A journey is permitted only when at least Rs 50 remains after deduction. Students and senior citizens receive a concession.

The question paper does not specify the concession percentage or the order in which concession and peak surcharge are applied. For a consistent implementation, this report assumes a 50% concession on the base fare first, followed by the peak surcharge. Money is stored in paise. Percentage calculations use integer arithmetic with half-up rounding. These assumptions should be confirmed with the faculty if a different rule is intended.

Peak interval boundaries are treated as inclusive at 08:00 and 17:00, and exclusive at 10:00 and 20:00. Thus 10:00 AM and 8:00 PM are off-peak.

GitHub and Execution Note

Create the required repository DSA01-<RegisterNumber>-Assignment1. Store each task in its own folder with source code, input data and output evidence. Use: g++ -std=c++17 -Wall -Wextra filename.cpp -o program. Execute each program and commit the work task by task. Timings in this report are not claimed as measurements; record timings from the system used for submission.

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

Ans: Object-Oriented Analysis of the Metro Fare System

The system has to calculate a journey fare, validate the passenger and time details, deduct the amount safely and retain the minimum balance. The main design decision is to assign each responsibility to a suitable class instead of allowing unrelated functions to modify shared data.

Object / Type	Important data	Behaviour / Responsibility	Reason for inclusion
SmartCard	cardId; balancePaise (private)	balance(); canTravel(fare); deduct(fare); recharge(amount)	Owns stored value and protects the minimum-balance rule.
Passenger	name; category	isEligibleForConcession()	Concession depends on passenger category.
Journey	zone; hour; minute; date	isPeakHour(); isValid()	Represents one trip and its time and zone.
Zone	Inner, Middle, Outer	Enumeration values	Restricts zone input to valid named choices.
FareTable	base fares; surcharge; concession; minimum balance	calculateFare(journey, passenger)	Keeps fare policy in one place.

Procedural and Object-Oriented Approaches

In a procedural design, a global balance variable and separate functions such as calculateFare(), checkBalance() and deductFare() perform the work. Every caller must remember to validate the balance before deduction. In an object-oriented design, SmartCard owns its balance and exposes a controlled deduct() operation. The class itself checks the rule, reducing the chance that another part of the program bypasses validation.

OOP Principle	Application in the metro system	Benefit
Encapsulation	Keep balancePaise private inside SmartCard.	Prevents uncontrolled balance modification.
Abstraction	Expose calculateFare() and deduct() rather than internal calculations.	Gate code uses simple operations.
Inheritance	StudentPassenger and SeniorPassenger may derive from Passenger.	Common passenger data is reused.
Polymorphism	Different DiscountRule implementations can override apply().	New discount policies can be introduced without rewriting the gate workflow.

Rules and Invariants

- After a successful journey, balance must be at least 5,000 paise (Rs 50).
- Fare must be selected only from the valid zone fare table.

- Peak surcharge applies only in the two stated time intervals.
- Invalid zones, hours, minutes and non-positive recharge amounts must be rejected.
- All money calculations must use integer paise to avoid binary floating-point rounding.
- One authoritative definition must exist for each fare and policy constant.

Alternative Concept and Complexity Analysis

A procedural version would use global variables for balance and fare constants, with free functions to calculate and deduct fares. Both procedural and object-oriented designs perform a fixed number of operations per journey, so time is O(1). Each card stores a fixed number of fields, so memory per card is O(1). When a fare rule changes, the procedural design may require checking every function that accesses the globals. The object-oriented design normally localises the change in FareTable or a policy class, making maintenance safer.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

Ans: Tokens and Constant Management

A token is the smallest meaningful unit recognised by the C++ compiler. The fare program uses the following five token categories.

Token category	Examples	Purpose
Keywords	class, enum, const, constexpr, if, else, return	Reserved language words.
Identifiers	SmartCard, balancePaise, calculateFare, MIN_BALANCE_PAISE	Programmer-defined names.
Constants / literals	5000, 20, true, 'A', "Denied"	Fixed numeric, Boolean, character and string values.
Operators	=, +, -, *, /, %, >=, &&, ||, +=, <<	Perform calculations, comparisons and stream operations.
Punctuators	{}, (), [], ;, , , ::	Group statements and separate declarations.

#define, const and constexpr

Feature	#define	const	constexpr
Processing	Preprocessor text replacement	Compiler declaration	Compiler declaration
Type checking	No declared type	Yes	Yes
Scope	Not governed by C++ block scope	Normal C++ scope	Normal C++ scope
Compile-time value	Text substitution	Not always compile-time	Compile-time constant when valid
Suitability	Avoid for typed fare data	Useful for read-only values	Best for fixed fare constants

constexpr is preferred for fixed fare values because it has a type, follows scope rules and is checked by the compiler. It can also be used where a compile-time constant is required.

```
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int CONCESSION_PERCENT = 50;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

enum versus enum class

A plain enum can convert to an integer implicitly and its enumerator names may leak into the surrounding scope. enum class keeps values scoped, for example Zone::Outer, and does not implicitly convert to int. This reduces accidental mixing of zone values with ordinary numbers.

Identifier naming rules

- An identifier may contain letters, digits and underscores, but cannot begin with a digit.
- C++ identifiers are case-sensitive and cannot be reserved keywords.
- Use PascalCase for class and type names, camelCase for functions and variables, and UPPER_SNAKE_CASE for constants.
- Include units in money-related names, such as balancePaise and MIN_BALANCE_PAISE.

Alternative Concept and Complexity Analysis

Using #define macros would remove type checking and normal scope behaviour. For example, a macro can substitute text in unintended expressions unless parentheses are carefully added. A constexpr value normally requires no separate run-time storage when optimised, just as a macro does not. The fare lookup and arithmetic remain O(1), so running time is not meaningfully changed. constexpr is safer and easier to debug.

TASK 3 – Choose Data Types and Check Type Compatibility (4 Marks)

Ans: Data Type Selection

Value	Suggested type	Reason
Fare and balance	long long (paise)	Exact integer arithmetic and large range.
Peak / concession flag	bool	Only true or false is needed.
Hour and minute	int	Small bounded whole-number values.
Zone	enum class Zone	Only named valid zones are allowed.
Journey count	long long or std::size_t	Can represent millions of records.
Passenger name / card ID	std::string	Variable-length text.
Percentage	int	Whole-number percentage constants.

Typical sizes on a common 64-bit g++ platform are: bool 1 byte, char 1 byte, int 4 bytes, unsigned int 4 bytes, long long 8 bytes, float 4 bytes and double 8 bytes. Exact sizes are implementation-dependent; confirm them using sizeof on the compiler used for submission.

```
#include <iostream>
#include <climits>
int main() {
   std::cout << sizeof(bool) << ' ' << sizeof(char) << ' '
             << sizeof(int) << ' ' << sizeof(long long) << ' '
             << sizeof(float) << ' ' << sizeof(double) << '\\n';
   std::cout << INT_MIN << ' ' << INT_MAX << '\\n';
   std::cout << LLONG_MIN << ' ' << LLONG_MAX << '\\n';
}
```

Why integer paise is preferred

A double stores most decimal fractions as approximations in binary. Repeated additions and comparisons can therefore produce tiny errors. A fare of Rs 10.80 is represented as 1,080 paise, which is exact in an integer type. Integer paise also makes the Rs 50 minimum-balance comparison reliable.

Type compatibility

- bool and char values are promoted to int in many arithmetic expressions; therefore true + 2 evaluates to 3.
- If an int and double are combined, the int is converted to double before the operation.
- Integer division discards the fractional part: 7 / 2 is 3, while 7 / 2.0 is 3.5.
- Mixing signed and unsigned integers can convert the signed operand to unsigned, producing surprising comparisons.

Range and network total

A typical 32-bit int has a maximum of 2,147,483,647. At three million journeys, even a modest total of 3,600 paise per journey becomes 10,800,000,000 paise, which exceeds INT_MAX. A signed long long has a much larger range (at least 64 bits on the target platform) and is suitable for the aggregate.

Alternative Concept: double rupees

With double rupees, the program would store values such as 10.0 and 30.0 and calculate a surcharge using multiplication by 1.20. Addition is O(1) and both double and long long commonly occupy 8 bytes. However, decimal fractions such as 0.10 are not generally exact in binary. Summing Rs 0.10 repeatedly can differ slightly from the mathematically exact decimal total. Integer paise avoids this issue entirely. The number of additions is O(n) for n journeys in either representation.

TASK 4 – Control Implicit Conversions (4 Marks)

Ans: Conversion Analysis and Output Prediction

Implicit conversion occurs when C++ automatically changes a value from one type to another. These conversions are convenient but can cause incorrect calculations if their effects are ignored.

Expression / statement	Conversion	Result or risk
double c = a / b;	Integer division happens before conversion to double.	7 / 2 becomes 3, then 3.0.
static_cast<double>(a) / b	Explicit int-to-double conversion.	7.0 / 2 becomes 3.5.
char ch = 'A' + 2;	char is promoted to int for addition, then converted to char.	ch contains 'C'.
ch + 1	char is promoted to int.	68 is printed for ASCII-compatible execution.
bool flag = 5;	Non-zero int converts to true.	Printed as 1 or true.
s < u where s=-1, u=1u	s is converted to unsigned int.	Comparison is false on typical 32-bit unsigned int.

Predicted output

```
3 3.5
C 68
1 true
s >= u
```

The predicted output is also the expected output when compiled on a conventional system using the stated C++ rules. The first line demonstrates integer division. The second line demonstrates char-to-int promotion. The third line shows integer-to-bool conversion and boolalpha formatting. The final line shows the signed-to-unsigned conversion.

Narrowing and explicit casts

Brace initialisation rejects narrowing conversions at compile time. For example, int amount{3.7}; is ill-formed because a fractional double cannot be safely stored in an int. static_cast<double>(a) / b is appropriate when fractional division is intended. For money, the better correction is to use integer paise rather than convert money to double.

Safer illustrative code

```
int whole{7};
int divisor{2};
double exactQuotient = static_cast<double>(whole) / divisor;
bool enabled{true};
// int invalid{3.7}; // compile-time narrowing error
```

Alternative Concept and Complexity Analysis

If conversions were written explicitly and brace initialisation were used, the program would be more verbose but easier to audit. Built-in casts generally compile to the same operations as the corresponding implicit conversions, so time and memory are effectively unchanged (O(1) per conversion). Brace

initialisation catches narrowing errors during compilation rather than allowing a truncated value to reach run time.

TASK 5 – Write Correct Expressions with the Right Precedence (4 Marks)

Ans: Expressions, Precedence and Associativity

Fare expressions

```
int minutes = hour * 60 + minute;
bool peak = (minutes >= 8 * 60 && minutes < 10 * 60) ||
        (minutes >= 17 * 60 && minutes < 20 * 60);

// Apply 50% concession first, then 20% peak surcharge.
// Half-up rounding is used for percentage calculations.
if (concession)
   fare -= (fare * CONCESSION_PERCENT + 50) / 100;
if (peak)
   fare += (fare * PEAK_SURCHARGE_PERCENT + 50) / 100;
```

Multiplication is evaluated before addition in the minute calculation. Relational comparisons are evaluated before logical AND, and logical AND before logical OR. Parentheses make the two peak windows easy to identify. The compound assignment operators += and -= update the left operand using the calculated right-hand expression.

Increment operators and negative division

x++ returns the old value and increments x afterwards. ++x increments first and returns the new value. The expression i = i++ should be avoided because it is confusing and has historically had undefined or version-dependent sequencing behaviour; write i++ when only an increment is intended.

Expression	C++ result	Reason
10 / 3	3	Integer division truncates toward zero.
10 % 3	1	Remainder after integer division.
-10 / 3	-3	Truncation is toward zero.
-10 % 3	-1	Remainder has the sign of the left operand.

Predicted output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: 5 + (3 × 2) = 11, while (5 + 3) × 2 = 16. Line 2: x++ contributes 5 and changes x to 6; ++y changes y to 4 and contributes 4, so z is 9. Line 3: x is greater than y, so the conditional expression returns 6; (6 % 4) × 2 equals 4. Line 4 demonstrates integer division and remainder. Line 5 adds loop values 0, 3, 6 and 9, giving 18.

Alternative Concept: peak lookup table

```
bool peakHour[24] = {};
for (int h = 0; h < 24; ++h)
   peakHour[h] = (h >= 8 && h < 10) || (h >= 17 && h < 20);
bool peak = peakHour[hour];
```

Both the Boolean expression and the lookup table provide O(1) checking. The table uses 24 Boolean entries (commonly 24 bytes) but must be initialised and updated if the rule changes. The direct expression uses no additional table storage and handles minute-level boundaries more naturally.

TASK 6 – Design the Control Structures (4 Marks)

Ans: Decisions, Loops and Fare Algorithm

Control structure	Use in this system	Reason
switch	Select base fare using Zone.	Zone is a fixed set of alternatives.
if–else	Check peak time, concession and balance.	Conditions involve ranges and multiple tests.
Conditional operator ?:	Choose a short display label such as Allowed / Denied.	Suitable for a simple two-way value.
for loop	Process a known list of journeys.	The number of journeys is known.
Range-based for	Visit every Trip in a container or array.	Avoids manual index handling.
while loop	Read journeys until input ends.	Useful when the number of records is unknown.

Pseudocode

```
ALGORITHM CalculateFare
INPUT: zone, hour, minute, concession
OUTPUT: fare in paise
BEGIN
   SELECT base fare using zone
   IF zone is invalid THEN
      RETURN INVALID
   END IF
   peak <- (08:00 <= time < 10:00) OR (17:00 <= time < 20:00)
   IF concession THEN
      base <- base - rounded(base * 50 / 100)
   END IF
   IF peak THEN
      base <- base + rounded(base * 20 / 100)
   END IF
   RETURN base
END

ALGORITHM ProcessJourneys
INPUT: journey list, smart-card balance
BEGIN
   FOR EACH journey
      IF journey data is invalid THEN
         CONTINUE
      END IF
      fare <- CalculateFare(journey)
      IF balance - fare >= 5000 THEN
         deduct fare from balance
         display fare and balance
      ELSE
         display "Denied"
      END IF
   END FOR
END
```

Completed TODO code

```
long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak)
        base += (base * PEAK_SURCHARGE_PERCENT + 50) / 100;
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
```

For the supplied main() and starting balance of Rs 100.00, the expected output is:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

At exactly 10:00 AM and 8:00 PM the peak condition is false. At exactly 8:00 AM and 5:00 PM it is true.

Alternative Concept: array lookup

An array such as constexpr long long FARE[] = {1000, 2000, 3000}; allows direct O(1) indexing, provided the enum values map safely to array indices and are validated. A switch is explicit and can warn about missing enum cases. Adding a fourth zone requires a new enum value and fare entry in either design; an array-based table makes data-driven expansion easier.

TASK 7 – From Procedural to Object-Oriented (4 Marks)

Ans: Two Implementations and Comparison

Procedural version

```
long long cardBalance = 10000;
constexpr long long MIN_BALANCE = 5000;

bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= MIN_BALANCE;
}
bool deductFare(long long& balance, long long fare) {
    if (!canTravel(balance, fare)) return false;
    balance -= fare;
    return true;
}
```

Object-oriented version

```
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_BALANCE_PAISE = 5000;
public:
    explicit SmartCard(long long openingBalance)
        : balancePaise(openingBalance) {}

    long long balance() const { return balancePaise; }

    bool canTravel(long long fare) const {
        return fare >= 0 &&
               balancePaise - fare >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }

    bool recharge(long long amount) {
        if (amount <= 0) return false;
        balancePaise += amount;
        return true;
    }
};
```

The procedural version can enforce the rule if every function is used correctly, but any code with access to the global balance can modify it directly. In the class version, balancePaise is private. Other parts of the program must use deduct() or recharge(), so the class can reject an invalid operation before changing the balance.

Alternative Concept: public balance member

If balance were public, any code could assign a negative value or subtract a fare without checking the Rs 50 reserve. The execution time and memory use would not materially change: both remain O(1) per operation and O(1) per card. The difference is correctness and maintainability, not performance. A private member protects the balance invariant.

TASK 8 – Read and Print Data Safely (4 Marks)

Ans: Input Validation and Output Formatting

The gate program should reject invalid input instead of silently treating it as a valid zone or time. When extraction with cin fails, clear the stream state and discard the rest of the invalid line.

```
#include <iostream>
#include <iomanip>
#include <limits>

int readInteger(const char* prompt) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value) return value;

        std::cout << "Invalid input. Enter a whole number.\\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
    }
}

void printFare(long long paise) {
    std::cout << "Fare Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0') << paise % 100
              << std::setfill(' ') << '\\n';
}
```

cin.fail() can be checked explicitly after extraction. cin.clear() removes the failure state, while ignore() discards the invalid characters so the next read can proceed. setw(2) sets the width for the next output field and setfill('0') pads the paise part with a leading zero. fixed and setprecision(2) are useful when displaying a double, but integer paise formatting is safer for money.

Alternative Concept: getline() and stoi()

```
std::string line;
if (std::getline(std::cin, line)) {
    try {
        std::size_t used = 0;
        int hour = std::stoi(line, &used);
        if (used != line.size()) throw std::invalid_argument("extra text");
        // validate hour range here
    } catch (const std::exception&) {
        std::cout << "Invalid hour input\\n";
    }
}
```

For a line of length L, reading and converting it takes O(L) time. getline stores the line in a string, requiring O(L) buffer space. Direct cin >> is convenient for token-based input and recovers through clear()/ignore(). getline() with stoi() gives control over the complete line and can reject trailing characters, but requires exception handling and explicit checks.

TASK 9 – Implement and Test the Fare Logic (4 Marks)

Ans: Test Plan and Validation

The following tests use the stated fare rules and the working assumption of a 50% concession applied before the peak surcharge. A fare is calculated in paise, and a card is accepted only when the remaining balance is at least 5,000 paise.

Test	Input / Condition	Expected result	Purpose
T1	Inner, 07:30, regular	Rs 10.00	Normal off-peak fare.
T2	Middle, 08:00, regular	Rs 24.00	Peak boundary start.
T3	Outer, 10:00, regular	Rs 30.00	Peak boundary end is excluded.
T4	Inner, 17:00, student	Rs 6.00	Concession then peak surcharge.
T5	Outer, 20:00, senior	Rs 30.00	20:00 is off-peak; concession assumed 50% would give Rs 15.00
T6	Unknown zone value	Rejected	Invalid zone must not be priced.
T7	Recharge = -100 paise	Rejected	Recharge must be positive.
T8	Hour entered as letters	Input error handled	Stream must recover or request valid input.
T9	Balance Rs 60, fare Rs 10	Accepted; remaining Rs 50	Minimum balance equality boundary.

For T5, an Outer-zone senior journey at 20:00 receives the 50% concession and no peak surcharge, so the expected fare is Rs 15.00.

Five-journey hand calculation

Journey	Zone / time / category	Fare	Running total
1	Inner, 07:30, regular	Rs 10.00	Rs 10.00
2	Middle, 08:30, regular	Rs 24.00	Rs 34.00
3	Outer, 10:00, regular	Rs 30.00	Rs 64.00
4	Inner, 17:30, student	Rs 6.00	Rs 70.00
5	Middle, 20:00, senior	Rs 10.00	Rs 80.00

For journey 4, the concession reduces Rs 10 to Rs 5; the 20% surcharge on Rs 5 gives Rs 6. For journey 5, the concession reduces Rs 20 to Rs 10 and there is no peak surcharge. Total fare for the five journeys is Rs 80.00.

Alternative Concept: table-driven tests

Instead of writing separate checks for every test, store each test input and expected result in a test-case array and loop through it. For t test cases, the time is O(t), and the table requires O(t) space. A new boundary case can be added as one new row without changing the test loop.

TASK 10 – Compare Expectations with Execution (4 Marks)

Ans: Expected and Actual Behaviour

The question requires comparison at 20,000, 500,000 and 3,000,000 journeys. The exact total depends on the actual journey mix and fares. To make the type-range comparison reproducible, the following stress test assumes a fixed fare of 3,600 paise (Rs 36) for every journey. Replace this assumption with the fares generated by the actual test dataset when submitting execution evidence.

Journeys	Total at 3,600 paise each	Fits in 32-bit signed int?	Fits in long long?
20,000	72,000,000 paise	Yes	Yes
500,000	1,800,000,000 paise	Yes, narrowly	Yes
3,000,000	10,800,000,000 paise	No – overflow	Yes

A 32-bit signed int has a maximum value of 2,147,483,647. Therefore, the three-million-journey stress-test total exceeds its range. A 64-bit long long can represent this total safely. A double can represent these whole-number paise totals exactly at this scale, but using double rupees for fractional fares can introduce rounding differences.

Reproducible comparison code

```
#include <iostream>
#include <iomanip>
int main() {
   const long long farePaise = 3600;
   const long long counts[] = {20000, 500000, 3000000};
   for (long long n : counts) {
      long long exactPaise = n * farePaise;
      double rupees = 36.0 * static_cast<double>(n);
      std::cout << n << " journeys: "
                << exactPaise << " paise, Rs "
                << std::fixed << std::setprecision(2)
                << rupees << '\\n';
   }
}
```

Compile with -Wall -Wextra. Investigate warnings such as signed/unsigned comparison, unused variables, narrowing conversions and missing return statements. The precise warning text depends on the code and compiler version. Record the actual compiler output and timing results rather than inserting unverified measurements.

Alternative Concept: accumulation types

Type	At 3,000,000 × 3,600 paise	Assessment
int	Exceeds 2,147,483,647	Signed overflow; incorrect result.
long long	10,800,000,000 paise	Safe for this workload.
double	Can represent this integer total at this magnitude	Suitable for approximate scientific values, but not preferred for exact currency.

Each addition is O(1) for all three types. Each variable uses O(1) space. Typical storage is 4 bytes for int and 8 bytes for long long and double. The choice of long long paise gives exact currency arithmetic and sufficient range.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Ans: Final Design Recommendation

The recommended solution is to retain the object-oriented structure and use integer paise for every monetary value. SmartCard should own and protect the balance, Passenger should represent concession eligibility, Journey should validate zone and time, and FareTable should hold the fare policy.

Design element	Recommendation	Reason
Money type	long long paise	Exact arithmetic and sufficient range.
Zone	enum class Zone	Prevents accidental integer/zone mixing.
Constants	constexpr values in one header	Single source of truth and compile-time checking.
Card	SmartCard with private balance	Enforces the minimum remaining balance.
Fare policy	FareTable / policy class	Centralises surcharge and concession rules.
Input	Validated numeric input	Rejects malformed values and invalid ranges.
Testing	Table-driven boundary and normal tests	Makes new test cases easy to add.

Fare revision process

A fare revision should update the central fare configuration, followed by recompilation, automated tests and deployment. No fare value should be repeated in unrelated functions. Boundary tests must be rerun for 08:00, 10:00, 17:00 and 20:00, as well as the minimum-balance condition.

Limitations

- The initial design assumes a fixed concession percentage and a specific order of concession and surcharge; the business owner must confirm these rules.
- A compiled-in fare table requires rebuilding and redeploying the software after a fare revision.
- The design needs additional policy objects to support weekend discounts, monthly passes and more zones cleanly.
- Persistent card updates should be transactional in a real metro system so a power failure cannot partially deduct a fare.

Alternative Concept: configuration file

A configuration file can store the fare values and policy percentages and be loaded when the gate starts. Reading z zone fares takes O(z) startup time and O(z) storage. It avoids recompilation for simple fare revisions, but the file must be validated, versioned and protected from unauthorised changes. A compiled constexpr table has almost no startup cost and catches some mistakes at compile time, but each revision requires rebuilding and redeployment. For a production network, a validated, signed configuration with safe fallback values may be more flexible.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The metro introduces a fourth Airport zone with a base fare of Rs 60, a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that permits unlimited journeys in two selected zones.

A. Impact Analysis (2 Marks)

Original element	Change required	Reason
Zone enum	Add Zone::Airport.	A fourth valid travel zone is introduced.
Fare constants	Add 6,000 paise for Airport.	The new zone has a Rs 60 base fare.
Fare selection switch	Add Airport case or replace switch with a validated table.	All four zones must be mapped.
Peak expression	Retain the existing time boundaries.	Peak hours are not changed.
Discount logic	Add weekend rule and make it mutually exclusive with peak surcharge.	Weekend discount cannot combine with peak surcharge.
Passenger / fare classes	Add a policy abstraction for discount rules.	Keeps discount decisions separate from zone lookup.
SmartCard / Pass	Add a MonthlyPass class with two permitted zones and validity period.	Unlimited travel is a different entitlement from stored value.

B. Evaluation of the Original Design (2 Marks)

The original switch can support Airport by adding another case, but every new zone increases the amount of fare-selection code. A central indexed fare table is easier to extend when zone identifiers are mapped safely to array indices. SmartCard remains useful for stored-value users, but a monthly pass should not be represented as an ordinary balance because its entitlement is based on validity and permitted zones.

Worked example: Airport base fare is Rs 60. On a weekday peak journey, the fare is Rs 72 before any passenger concession under the initial policy assumption. On a weekend, the 10% discount applies and the peak surcharge is not added, giving Rs 54. The weekend rule therefore takes precedence over the peak surcharge.

C. Adaptation / Revised Design (2 Marks)

The revised design retains SmartCard but modifies the fare policy and adds MonthlyPass. The following compact implementation shows the revised zone table and mutually exclusive weekend/peak logic.

```
#include <array>
#include <optional>

enum class Zone { Inner, Middle, Outer, Airport };
enum class PassengerType { Regular, Student, Senior };

constexpr std::array<long long, 4> BASE_PAISE{
   1000, 2000, 3000, 6000
};
constexpr long long MIN_BALANCE_PAISE = 5000;

long long roundedPercent(long long amount, int percent) {
   return (amount * percent + 50) / 100;
}

std::optional<long long> revisedFare(
   Zone zone, bool peak, bool weekend, PassengerType passenger) {

   auto index = static_cast<std::size_t>(zone);
   if (index >= BASE_PAISE.size()) return std::nullopt;

   long long amount = BASE_PAISE[index];

   if (passenger != PassengerType::Regular)
      amount -= roundedPercent(amount, 50);

   if (weekend) {
      amount -= roundedPercent(amount, 10);
      // Weekend discount does not combine with peak surcharge.
   } else if (peak) {
      amount += roundedPercent(amount, 20);
   }
   return amount;
}

class MonthlyPass {
   int firstZone;
   int secondZone;
   int validDaysRemaining;
public:
   MonthlyPass(int a, int b, int days)
      : firstZone(a), secondZone(b), validDaysRemaining(days) {}

   bool allows(Zone z) const {
      int value = static_cast<int>(z);
      return validDaysRemaining > 0 &&
            (value == firstZone || value == secondZone);
   }
};
```

The example assumes the passenger concession remains applicable before the weekend/peak rule. If faculty specifies a different concession interaction, update the policy function and its tests. In production code, validate constructor arguments for MonthlyPass, define the pass start/end dates, and handle renewals and expired passes.

D. Evidence-Based Defence (2 Marks)

Test case	Expected fare / result	What it verifies
Airport weekday off-peak, regular	Rs 60.00	New zone base fare.
Airport weekday peak, regular	Rs 72.00	20% peak surcharge.
Airport weekend, regular	Rs 54.00	10% weekend discount.
Airport weekend during peak time	Rs 54.00	Weekend discount excludes peak surcharge.
Monthly pass: selected zone, valid	Allowed; no per-journey fare deduction	Unlimited entitlement in a permitted zone.
Monthly pass: unselected zone	Not allowed by pass	Zone restriction is enforced.
Monthly pass: expired	Not allowed by pass	Validity is enforced.

Decision: MODIFY the original design rather than discard it. SmartCard remains a suitable encapsulated stored-value object, while the fare policy should be extended with a validated table and a separate discount rule. MonthlyPass should be a distinct class because its behaviour differs from stored-value cards. This design keeps existing functionality while isolating the new requirements.

FINAL SUMMARY

The metro fare system is best implemented with typed constants, enum class for zones, integer paise for money, explicit validation and a SmartCard class that protects the minimum-balance rule. Control structures should be selected according to the decision being made: switch for a finite zone choice, if–else for ranges and policy checks, and loops for journey processing. The revised design should retain the card abstraction, extend the fare policy for the Airport zone and weekend discount, and introduce a separate MonthlyPass entitlement.

Submission Checklist

- Complete and execute all programs using the required C++ compiler.
- Create Task01 to Task12 folders in the GitHub repository.
- Include source files, input files, actual output screenshots and a README.md.
- Record compiler version and exact compile commands.
- Add at least one meaningful commit for each task.
- Replace illustrative test evidence with the actual results obtained on your computer.
- Confirm the concession percentage and discount order with the faculty before final submission.